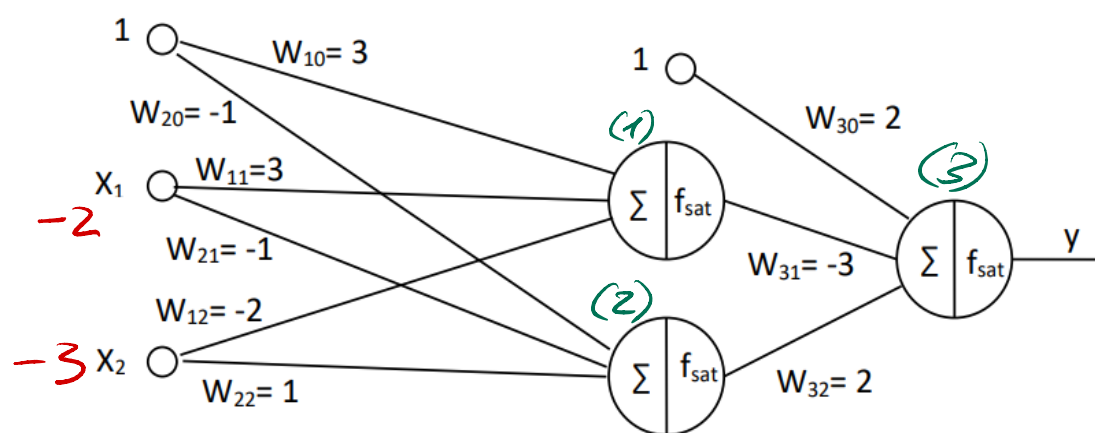




$$(1) \quad f_{\text{sat}}(1 \cdot 3 + (-1) \cdot (-2) + (-3) \cdot (-2)) = f_{\text{sat}}(3) = 0,6$$

$$(2) \quad f_{\text{sat}}(1 \cdot (-1) + (-1) \cdot (-2) + (-3) \cdot (1)) = f_{\text{sat}}(-2) = -0,4$$

$$(3) \quad f_{\text{sat}}(0,6 \cdot (-3) + (-0,4) \cdot 2 + 2) = -0,12 = y$$

$$\frac{\partial E}{\partial w_{11}} = \frac{\partial y}{\partial w_{11}} \frac{\partial E}{\partial y} = \frac{\partial y}{\partial w_{11}} (t - y)$$

$$\frac{\partial y}{\partial w_{11}} = f'_{\text{sat}}(-0,6) \cdot w_{31} \cdot f'_{\text{sat}}(3) \cdot (-2) = 0,2 \cdot (-3) \cdot 0,2 \cdot (-2) = 0,24$$

$$\Rightarrow \frac{\partial E}{\partial w_{11}} = 0,24 \cdot (1 + 0,12) = 0,2688$$

$$\Rightarrow w'_{11} = w_{11} - \eta \frac{\partial E}{\partial w_{11}} = 3 + 0,1 \cdot 0,2688 = 3,02688$$

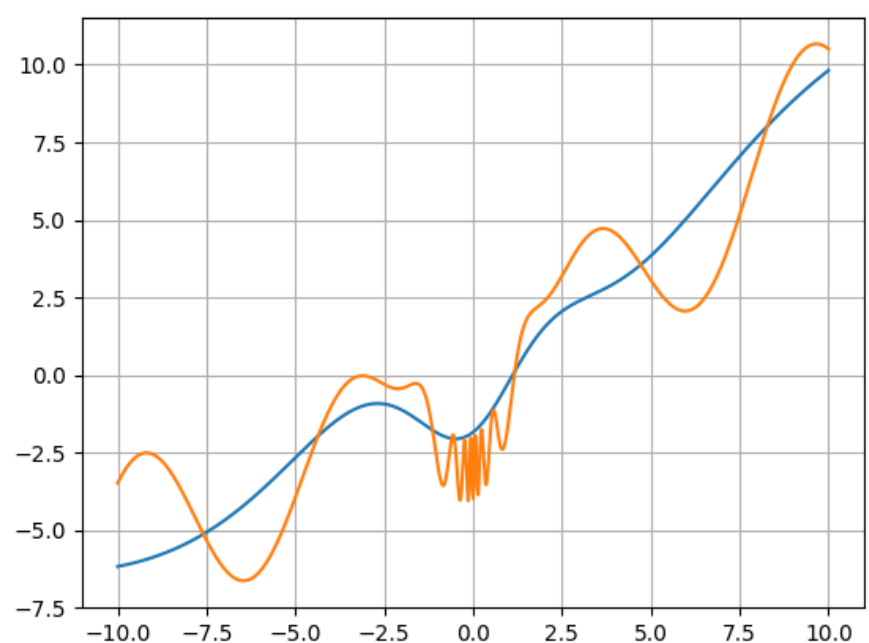
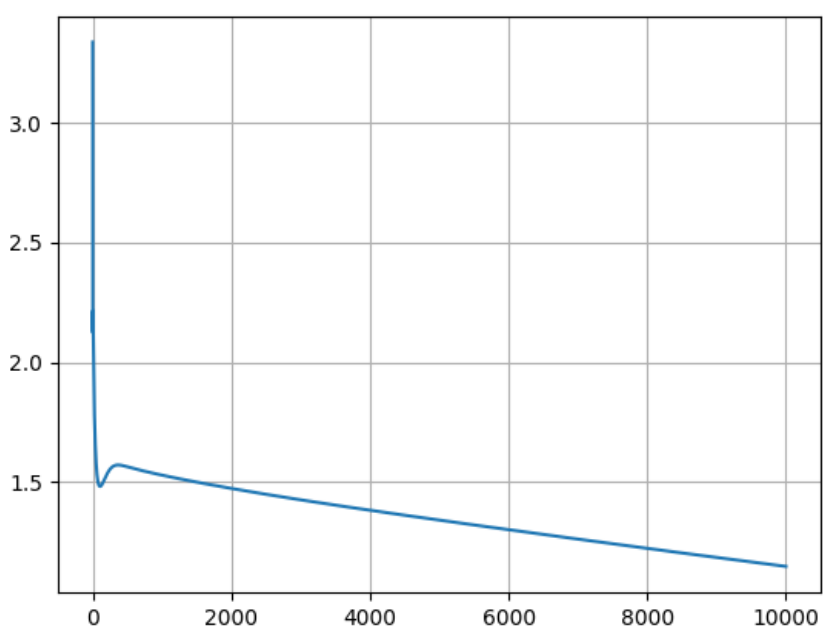
2.2 & 2.3

Wir haben einmal das NN und Backprop. selber implementiert (multi-layer-perception.py) und einmal mit pytorch gelöst (A2-3 Lauwa (...).py)

In der eigenen Implementierung wurde eine Lernrate von $\eta = 0.001$ verwendet.

2.2 Es wurden 10000 Trainingläufe durchlaufen und man sieht, dass das Netzwerk recht schnell ein Minimum findet.

Die sehr unlinearen Teile von der Target funktion bei $x=0$ kann das Netz nicht gut lernen. Eine angepasste Lernrate (Momentum beim Lernen z.B.) würde das Lernen stark beschleunigen.



2.3 Es wurden 600 Trainingläufe durchlaufen.

Das Netz findet erst keine guten Werte und der Fehler geht hoch und runter. Dies liegt, daran dass der Raum der Loss-Funktion nun viel komplizierter ist.

Der finale Fehler ist kleiner als, als nur das erste Layer trainiert wurde.

